

Multi-Declaration Z Constructs — Q2(d) Demo

txt2tex

May 19, 2026

Background

The constructs in this file all share a single grammatical shape: a Z paragraph that introduces two or more typed variables in one declaration list, with a conjunction predicate over their projections, optionally followed by a characteristic expression. Until the Q2(d) parser fix, none of them parsed natively — students wrote them as raw LaTeX. They now compile to fuzz-clean output.

$[ShipName, ClassName, BattleName, CountryName, Result]$

<i>Class</i>
<i>class</i> : <i>ClassName</i>
<i>country</i> : <i>CountryName</i>
<i>numGuns</i> : $\mathbb{N}$
<i>bore</i> : $\mathbb{N}$
<i>bore</i> ≤ 16

$PK(Class) = \{class\}$

<i>Ship</i>
<i>name</i> : <i>ShipName</i>
<i>class</i> : <i>ClassName</i>

$PK(Ship) = \{name\}$

<i>Outcome</i>
<i>ship</i> : <i>ShipName</i>
<i>battle</i> : <i>BattleName</i>
<i>result</i> : <i>Result</i>

$PK(Outcome) = \{ship, battle\}$

1 . WYSIWYG line breaks for algebra operators

Long relational-algebra chains can be broken at any operator. Natural newlines work; explicit continuation also works. Display position wraps in array; the chain reads top-to-bottom instead of running off the page.

$Project\{name, country\}(Class \otimes$
 $Ship \otimes$
 $Rename_{ship \rightarrow name}(Project\{ship\}(Restrict_{battle='Guadalcanal'}(Outcome))))$

2 . Multi - decl comprehension with conjunction predicate

This is the canonical DAT relational-calculus form for any query that crosses two or more relations. Before the fix it required a raw LATEX block; now it parses natively.

$$\{ s : Ship; c : Class; o : Outcome \mid s.class = c.class \wedge \\ o.ship = s.name \wedge \\ o.battle = 'Guadalcanal' \bullet \langle name == s.name, country == c.country, bore == c.bore \rangle \}$$

3 . Multi - decl quantifier with conjunction predicate

‘ $\forall s : T \bullet \forall c : U \bullet predicate.$ body’ works the same way — nested Quantifier nodes covering the chained declarations.

$$\forall s : Ship \bullet \forall c : Class \mid s.class = c.class \wedge c.bore \geq 16 \bullet c.numGuns > 8$$

4 . Multi - decl quantifier without characteristic expression

When the predicate alone suffices — no ‘.’ separator, no body expression — the comprehension defaults the characteristic tuple to the signature (Z RM §3.9).

$$\{ s : Ship; c : Class \mid s.class = c.class \wedge c.bore \geq 16 \}$$

5 . Multi - decl mu (definite description)

‘mu’ parses with multi-decl signatures the same way. The current generator emits nested ‘ $(\mu \dots \mid (\mu \dots))$ ’ — *semantically equivalent but not Spivey - canonical. The same special - case treatment lambda just received is a follow - up for mu. Single - decl mu (shown here) emits cleanly.*

$$bigGun == (\mu c : Class \mid c.bore = 16 \bullet c.numGuns)$$

6 . Multi - decl lambda — Spivey - canonical form

Lambdas of more than one variable now emit as a single λ token with semicolon - joined SchemaText, matching ZRM 3.12 and the

$$fleet == (\lambda s : Ship; c : Class \mid s.class = c.class \bullet s.name)$$

7 . Paren - wrap fix

Fuzz requires ‘ $(\lambda \dots \bullet \dots)$ ’ — *parentheses around every lambda expression, at any position. The generator previously only wrapped leveled predicates emitted bare ‘ λ ’, which fuzz rejected with Opening parenthesis expected at symbol λ . The fix : drop the ‘paren*
 $(\lambda x : \mathbb{N} \bullet x + 1)$